

Company Description

Scaler Retail Pvt. Ltd. is a rapidly expanding retail organization operating across multiple cities in India. The company manages a wide network of physical stores supported by regional warehouses and an integrated supply chain system, while simultaneously running a fast-growing online e-commerce platform. Scaler offers a diverse range of products including electronics, fashion, grocery, and home essentials, catering to both in-store and digital customers. With thousands of daily transactions occurring across its offline and online channels, the company generates significant volumes of operational and customer data. As the business scales, leadership aims to modernize its data infrastructure to enable real-time insights, centralized reporting, and improved decision-making across sales, inventory, supply chain, and customer experience domains.

Business Context

Every day, **Scaler Retail Pvt. Ltd.** handles thousands of transactions across its physical stores and online platform. Customers visit stores, browse products, make purchases, and complete payments through various modes such as UPI, cards, and cash. Simultaneously, online users browse the website or mobile application, add products to their carts, place orders, and complete digital payments.

With each interaction, multiple data points are generated — new customers register, orders are created, payments are processed, inventory levels are updated, and shipment statuses change. In physical stores, POS systems record transactional data, while warehouse systems track stock movement and replenishment. On the online platform, clickstream data captures user behavior such as product views, searches, and cart additions.

As the business grows, the volume, velocity, and variety of data continue to increase. The organization now requires a centralized and scalable data platform that can ingest, process, and analyze this data efficiently to support reporting, operational monitoring, and strategic decision-making.

Dataset format

FileSource:

<https://drive.google.com/drive/u/0/folders/1oPBELdKx2gcZ6Uamur45729EXlt7j716>

1. **Customers.csv** :- Represents customer registrations from both online and offline channels.

Column Name	Data Type	Description
customer_id	STRING	Unique customer identifier
first_name	STRING	Customer first name

last_name	STRING	Customer last name
email	STRING	Email address
phone_number	STRING	Contact number
city	STRING	Customer city
state	STRING	State
signup_date	DATE	Date of registration
customer_type	STRING	Online / Offline / Both
is_active	String	If customer is active or not

2. **Stores.csv** :- Represents physical retail store details.

Column Name	Data Type	Description
store_id	STRING	Unique store ID
store_name	STRING	Store name
city	STRING	City

state	STRING	State
region	STRING	North / South / East / West
opening_date	DATE	Store opening date
store_type	String	Type of store

3. **Products.csv:-** Represents product master data.

Column Name	Data Type	Description
product_id	STRING	Unique product ID
product_name	STRING	Product name
category	STRING	Electronics / Fashion / Grocery / Home
sub_category	STRING	Sub-category
brand	STRING	Brand name
price	DOUBLE	Selling price
cost_price	DOUBLE	Procurement cost

launch_date	DATE	Product launch date
is_active	string	

4. **inventory.csv:-** Tracks inventory levels at stores and warehouses.

Column Name	Data Type	Description
inventory_id	STRING	Unique record ID
product_id	STRING	Product reference
location_type	STRING	Store / Warehouse
location_id	STRING	Store ID or Warehouse ID
quantity_available	INT	Current stock
recorder_level	INT	
last_updated	TIMESTAMP	Last update time

5. **orders.csv (High Volume Fact Table)** :- Represents both online and offline transactions.

Column Name	Data Type	Description
-------------	-----------	-------------

order_id	STRING	Unique order ID
customer_id	STRING	Customer reference
order_channel	STRING	Online / Store
store_id	STRING	Nullable (only for store purchases)
order_source	STRING	
order_date	DATE	Date of order
order_timestamp	TIMESTAMP	Exact time
order_status	STRING	Placed / Shipped / Delivered / Cancelled
total_amount	DOUBLE	Total order value
payment_id	STRING	Payment reference

This table is ideal for partitioning by order_date.

6. Order_items.csv:- Represents line-level item details per order.

Column Name	Data Type	Description
order_item_id	STRING	Unique row ID

order_id	STRING	Order reference
product_id	STRING	Product reference
quantity	INT	Quantity purchased
item_price	DOUBLE	Price per unit
discount_amount	DOUBLE	Discount applied
line_total	Double	

7. Payments.csv :- Represents payment transactions.

Column Name	Data Type	Description
payment_id	STRING	Unique payment ID
order_id	STRING	Order reference
payment_method	STRING	UPI / Card / NetBanking / Cash
payment_status	STRING	Success / Failed
failure_reason	STRING	

payment_timestamp	TIMESTAMP	Payment time
amount_paid	DOUBLE	Amount paid

8. clickstream_logs.csv (For Streaming Simulation) :- Represents user activity on website/app.

Column Name	Data Type
event_id	STRING
customer_id	STRING
session_id	STRING
event_type	STRING (view / add_to_cart / purchase / search)
product_id	STRING
event_timestamp	TIMESTAMP
device_type	STRING (Mobile / Desktop)
location_city	STRING
page_url	string

referrer_source	string
-----------------	--------

Relationship

Child Table	FK	Parent
orders	customer_id	customers
orders	store_id	stores
orders	payment_id	payments
order_items	order_id	orders
order_items	product_id	products
payments	order_id	orders
inventory	product_id	products
inventory	location_id	stores
clickstream	product_id	products

clickstream	customer_id	customers
-------------	-------------	-----------

Problem statement 1:-

We are receiving this files on a weekly basis, your responsibility it as follows:-

- Use this file and load it to a specific folder of HDFS location
- These CSV files contains different dataset with different schema
- Create a database called as {your_login_username}_raw
- Create a hive table and load this data into these tables
- Ensure the data loss does not happen while loading this data into hive
- Identify the correct datatypes to store this schema and overwrite the schema definition given in the table
- Identify the correct business need and ensure the below
 - Table data loss should be prevented
 - Incase of data table deletion, data should be recoverable
 - Identify the correct optimisation technique to store the data
 - Like partitioning and bucketing if required
 - File type for better performance
 - Right column keys for partitioning and bucketing if needed
 - NULL and NOT NULL columns if applicable
 - Identify the bad data or duplicate data
 - Identify if you need to create a data model or you just want to dump the data

Problem statement 2:-

Data Transformation Using Spark (Raw → Staging Layer)

Objective

After successfully loading the raw CSV datasets into Hive tables, your next responsibility as a Data Engineer at **Scaler Retail Pvt. Ltd.** is to transform the raw data into a structured, analytics-ready staging layer using Apache Spark.

The raw layer contains data exactly as received from source systems. It may include:

- Null values
- Schema inconsistencies
- Duplicate records
- Data type mismatches
- Unstandardized formats
- Inconsistent business values

Your task is to clean, standardize, and transform this data using Spark and write the transformed output into a new Hive database called:

- Create a database called as {your_login_username}_staging

Note

Why Do We Need a Staging Layer?

In enterprise data architecture:

- Raw layer = immutable, source-aligned storage
- Staging layer = cleaned, validated, standardized data
- Curated layer = business-aggregated and analytics-ready

The staging layer acts as:

- A controlled transformation zone
- A validation checkpoint
- A foundation for downstream aggregations

Tasks to Be Performed

1. Create Staging Database

- {your_login_username}_staging
- They must not overwrite raw tables.

2. Read Raw Hive Tables Using Spark

Using Spark (on YARN), read from:

- {your_login_username}_raw.customers
- {your_login_username}_raw.products
- {your_login_username}_raw.orders
- {your_login_username}_raw.order_items
- {your_login_username}_raw.payments
- {your_login_username}_raw.inventory
- {your_login_username}_raw.clickstream_logs

They must use SparkSession with Hive support enabled

3. Perform Data Cleaning

You must implement the following transformations:

A. Customers Table operations

- Remove duplicate customer_id records
- Standardize email to lowercase
- Trim whitespace from names

- Handle null phone numbers
- Validate customer_type values

B. Orders Table

- Remove orders with negative total_amount
- Filter out invalid order_status values
- Ensure order_date is not null
- Convert timestamps properly
- Handle cancelled orders correctly

C. Payments Table

- Ensure payment amount matches order total (basic validation)
- Separate failed payments into a flag column
- Remove duplicate payment_id records

D. Order Items

- Ensure quantity > 0
- Remove records with null product_id
- Calculate derived column:
$$\text{item_total} = \text{quantity} * \text{item_price} - \text{discount_amount}$$

E. Inventory

- Ensure quantity_available >= 0
- Standardize location_type values
- Remove stale records if necessary

4. Business Enrichment

You must enrich fact tables:

- Join orders with customers to bring customer_city
- Join order_items with products to bring category
- Join payments with orders

This creates a more analytics-friendly dataset

5. Partition Strategy for Staging

You must write transformed data:

- Partition orders_staging by order_date
- Partition clickstream_staging by event_date (derived from timestamp)

This reinforces partition design thinking.

6. Write Back to Hive Staging Tables

You must:

- Use Spark to write data back to Hive
- Create managed OR external staging tables based on your understanding
- Use overwrite mode carefully
- Maintain schema consistency

Target tables:

- {your_login_username}_staging.customers_stg
- {your_login_username}_staging.orders_stg
- {your_login_username}_staging.order_items_stg
- {your_login_username}_staging.payments_stg
- {your_login_username}_staging.inventory_stg
- {your_login_username}_staging.clickstream_stg
- ...

2. Read staging Hive Tables Using Spark

Using Spark (on YARN), read from:

- {your_login_username}_staging.customers_stg
- {your_login_username}_staging.orders_stg
- {your_login_username}_staging.order_items_stg
- {your_login_username}_staging.payments_stg
- {your_login_username}_staging.inventory_stg

- `{your_login_username}_staging.clickstream_stg`

Design the Data Model (Star Schema)

Instead of normalized tables, we now create:

Fact Table

- `{your_login_username}_analytics.fact_sales_analytics`

Dimension Tables

- `{your_login_username}_analytics.dim_customer_analytics`
- `{your_login_username}_analytics.dim_product_analytics`
- `{your_login_username}_analytics.dim_store_analytics`
- `{your_login_username}_analytics.dim_date_analytics`

Fact Table: fact_sales

Derived from:

- `orders_stg`
- `order_items_stg`
- `payments_stg`

Schema

Column Name	Data Type	Description
sales_key	BIGINT	Surrogate primary key (generated in analytics layer)
order_id	STRING	Business order identifier
customer_key	INT	Surrogate key from <code>dim_customer</code>
product_key	INT	Surrogate key from <code>dim_product</code>

store_key	INT	Surrogate key from <code>dim_store</code> (nullable for online)
date_key	INT	Surrogate key from <code>dim_date</code> (YYYYMMDD format)
order_channel	STRING	Online / Store
order_status	STRING	Placed / Shipped / Delivered / Cancelled
payment_status	STRING	Success / Failed
quantity	INT	Units purchased
gross_amount	DOUBLE	$\text{quantity} \times \text{item_price}$
discount_amount	DOUBLE	Discount applied
net_amount	DOUBLE	$\text{gross_amount} - \text{discount_amount}$
payment_method	STRING	UPI / Card / NetBanking / Cash
load_timestamp	TIMESTAMP	When this record was inserted into analytics layer

Read the STG tables from spark and perform the transformation to make it suitable according to the Fact table and load the data into the table. Ensure there should not be a duplicate record present in the table.

Dimension Tables

Read the STG tables from spark and perform the transformation to make it suitable according to the Fact table and load the data into the table. Ensure there should not be a duplicate record present in the table.

dim_customer

From customers_stg, Please choose the correct schema based on your understanding.

Add derived fields:

- customer_key (surrogate key)
- city
- state
- customer_segment (derived from spending)
- registration_year

dim_product

From products_stg, Please choose the correct schema based on your understanding.

Add:

- product_key
- category
- brand
- price_band (Low / Medium / Premium)

dim_store

From stores_stg, Please choose the correct schema based on your understanding.

Add:

- store_key
- region
- city
- store_type (Tier1 / Tier2 derived)

Transformations Required (Spark)

You must:

- Join staging tables
- Soe Calculate Fields:

- $\text{gross_amount} = \text{quantity} * \text{item_price}$
 - $\text{net_amount} = \text{gross} - \text{discount}$
- Join dimensions
- Generate surrogate keys
- Partition fact table by year/month

Problem to Solve

Top Analysis

- Top 10 Products by Revenue
- Top 10 Customers by Spending
- Top Performing Stores

Customer Insights

- Repeat Customer %
- Customer Lifetime Value (CLV)
- Average Order Value (AOV)
- Monthly Active Customers

Store Performance

- Revenue per Store
- Revenue per Region
- Store-wise Conversion (if clickstream added)

Hive Table Creation Strategy

Students must:

- Store fact table in Parquet
- Partition it accordingly
- Use Snappy compression

- Use ORC/Parquet for dimensions

For the assessment, Please create output files of each problem and store it in the HDFS folder called as business output. The folder name must be {your_login_username}_analytics_output/